MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Introduction to Machine Learning · Day 2

Not Fooling Yourself

Trieste · Paolo Muratore · Tuesday 10 November 2026

CC BY

Overview



Where we learn to tell whether a model really works

Yesterday's Recipe

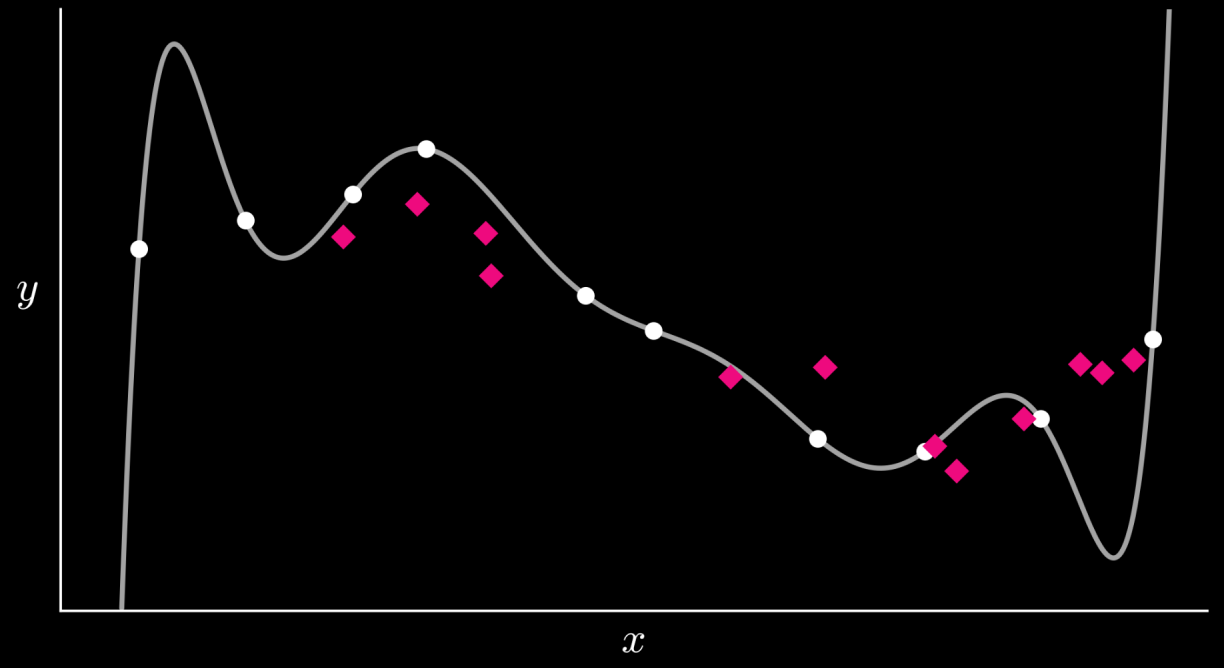
Data, model, loss, optimiser: yesterday we learned to fit

Data	$(x_i,\ y_i)$	inputs and their answers
Model	$f(x;\ \theta)$	a function with knobs
Loss		squared error, cross-entropy
Optimiser		gradient descent

Monday's promise: turn the knobs until the loss on our data is small

Is It a Good Model?

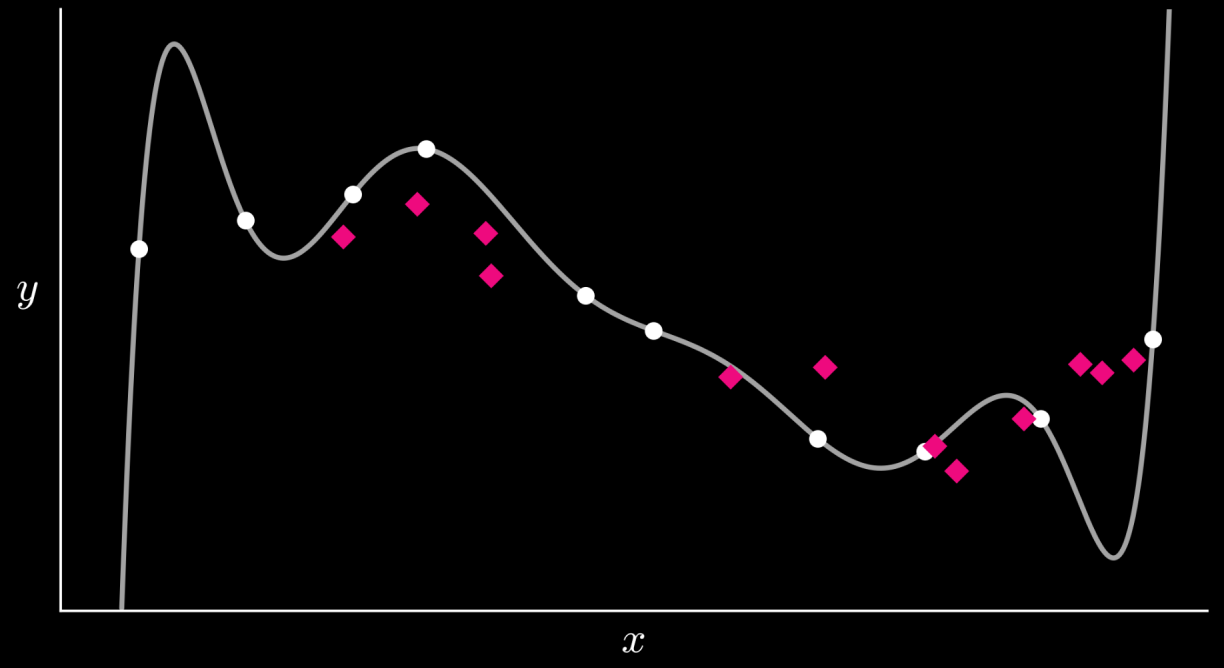
This curve passes through every training point: its training loss is zero



Would you trust its prediction for a new point?

Is It a Good Model?

This curve passes through every training point: its training loss is zero



Fitting the data you have is easy. Predicting the data you *don't* have is the point.

Two Kinds of Error

What we can measure is not what we care about

training error

on the data the model was fitted on

$$E_{\text{train}} = \frac{1}{N} \sum_{i \in \text{train}} \ell\big(y_i,\ f(x_i)\big) = 0.00$$

test error

on new data from the same source

$$E_{\text{test}} = \frac{1}{M} \sum_{j \in \text{test}} \ell\big(y_j,\ f(x_j)\big) \approx 0.50$$

the noise alone: 0.06

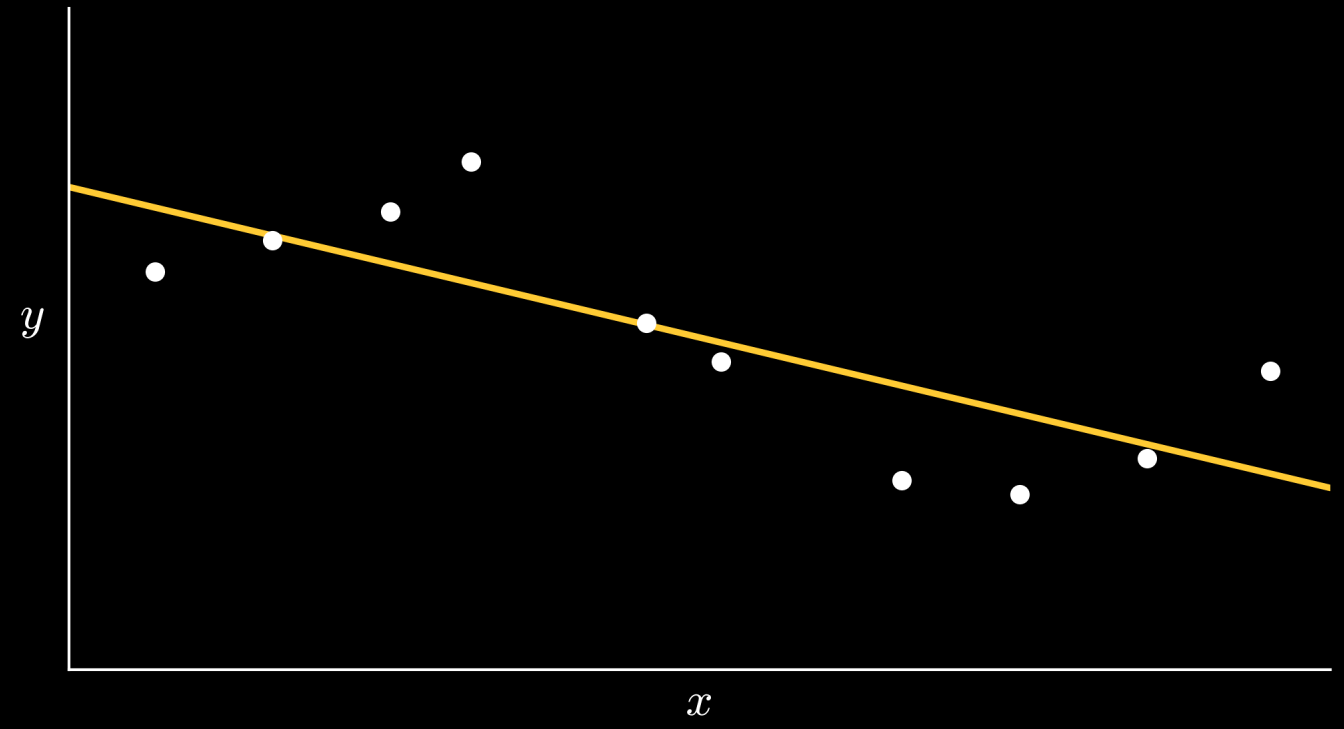
We want a small test error. A small training error can be a lie.

Too Simple, Too Flexible

Underfitting, overfitting, and the U-curve

Bending the Line

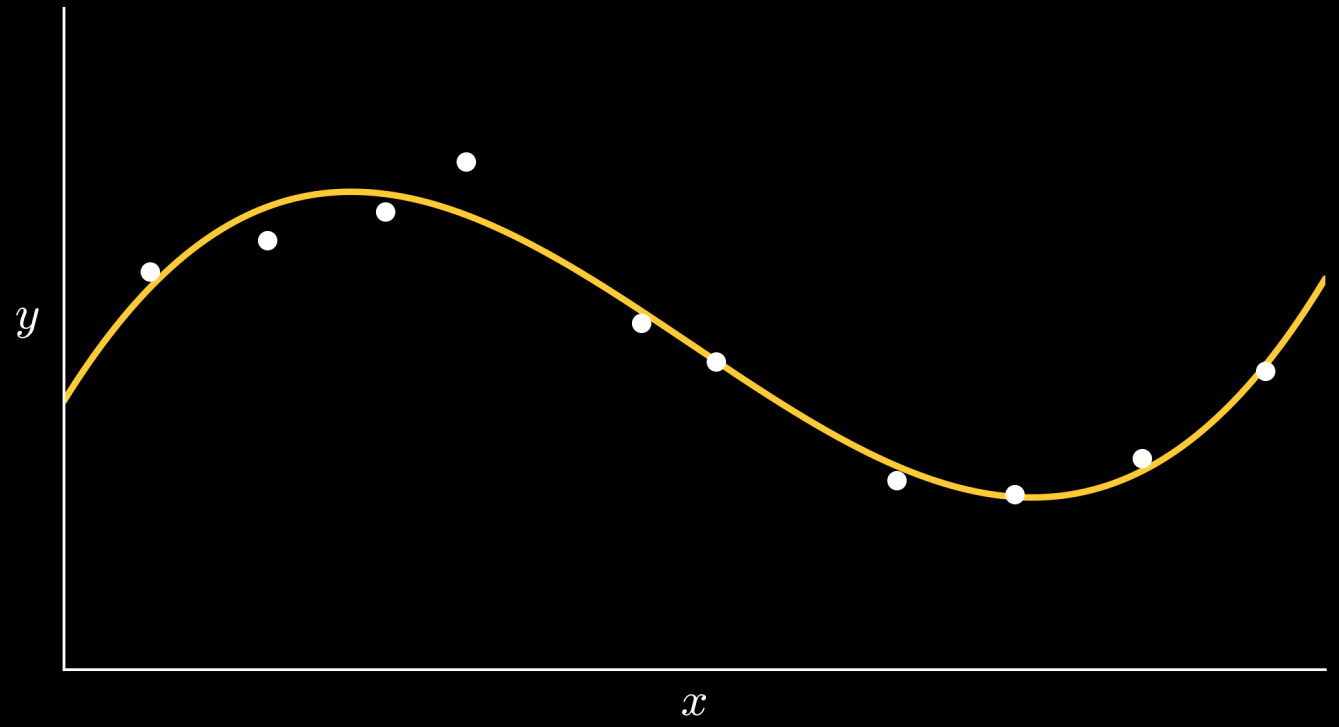
Add x^2, x^3, ... as features: still Monday's linear model, now a curve



$$\hat{y} = w_1\, x + b$$

Bending the Line

Add x^2, x^3, ... as features: still Monday's linear model, now a curve



$$\hat{y} = w_1\, x + w_2\, x^2 + w_3\, x^3 + b$$

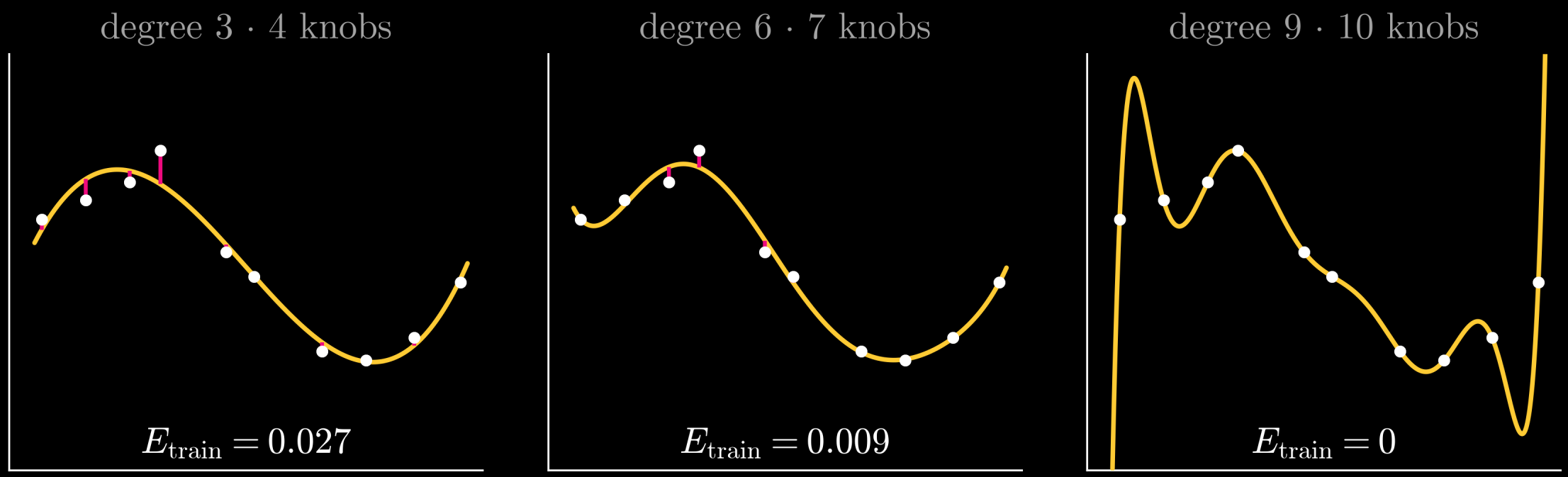
$$\hat{y} = \sum_{k=1}^{d} w_k\, x^k + b$$

degree $d \to d + 1$ knobs

the data matrix grows sideways: one column per power

One Knob per Point

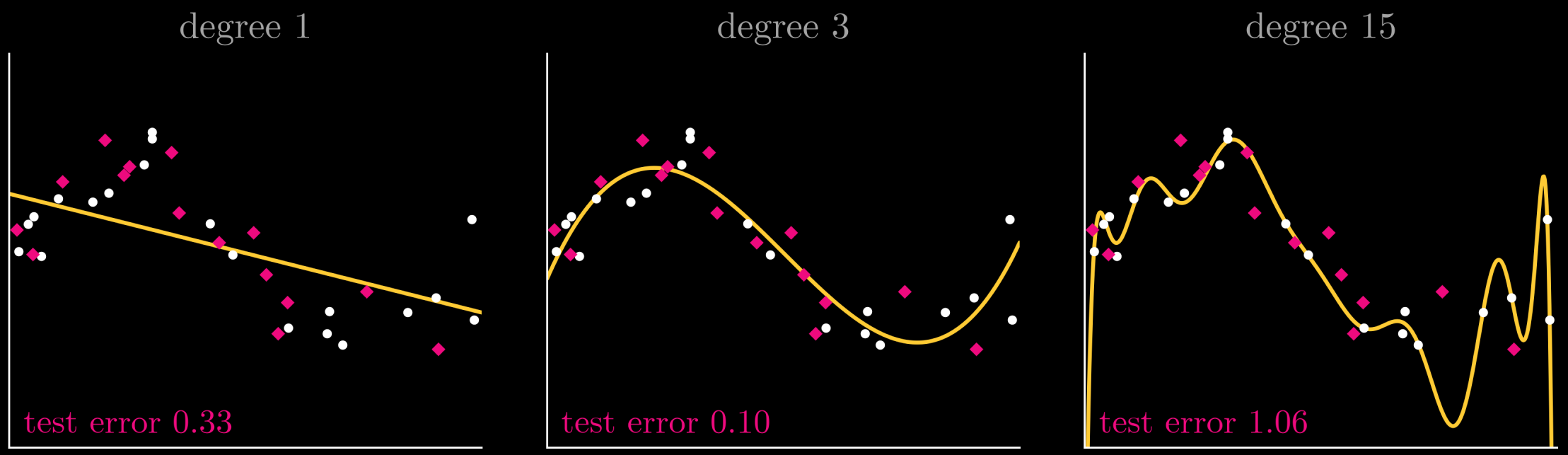
Your turn: how many knobs does a curve need to pass through ten points?



Zero training error was guaranteed, not earned:
with a knob per point, a model can memorise anything, noise included.

Three Models, One Dataset

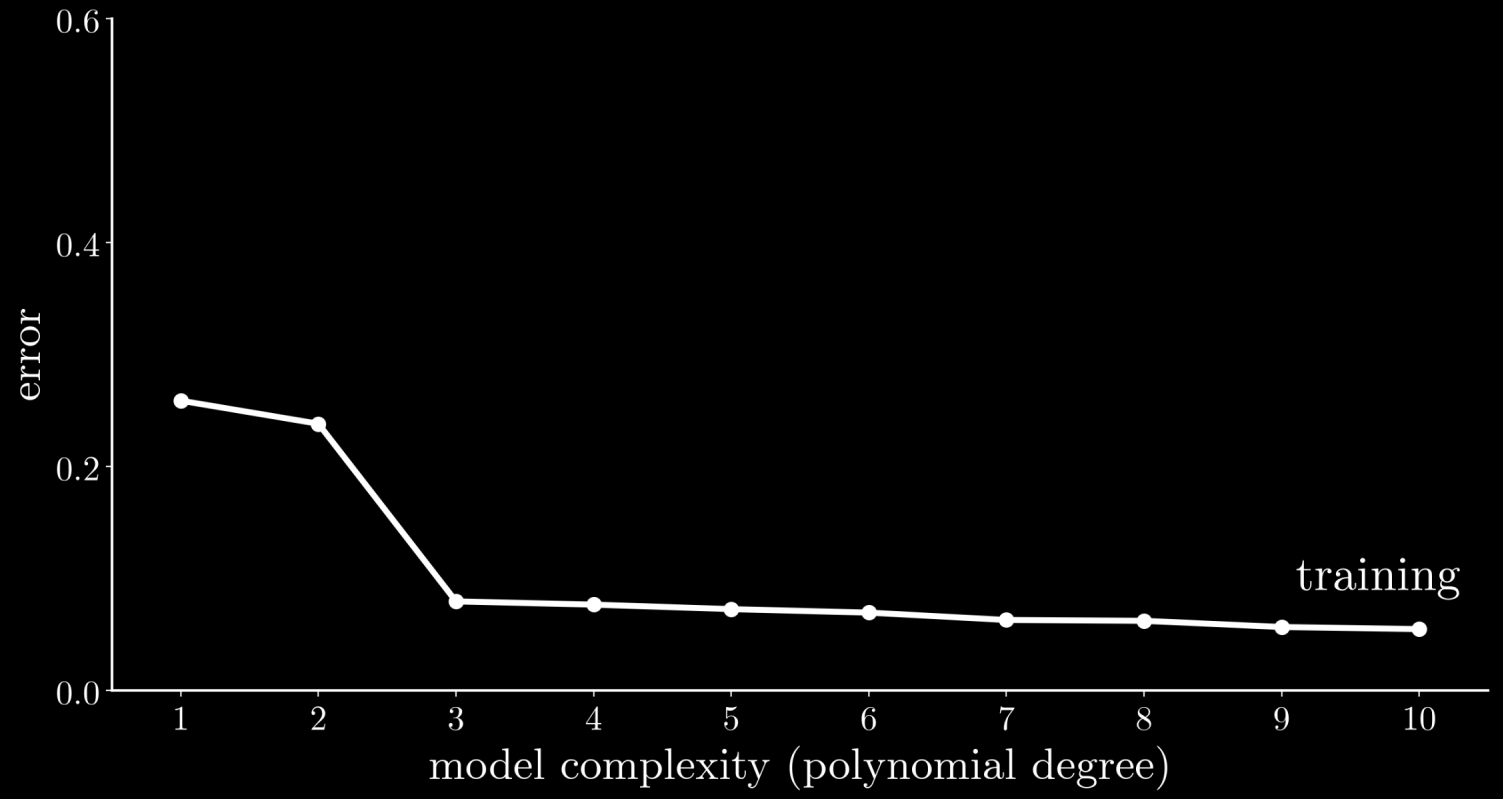
Your turn: which model will predict new points best?



too simple | *about right* | *too flexible*

The U-Curve

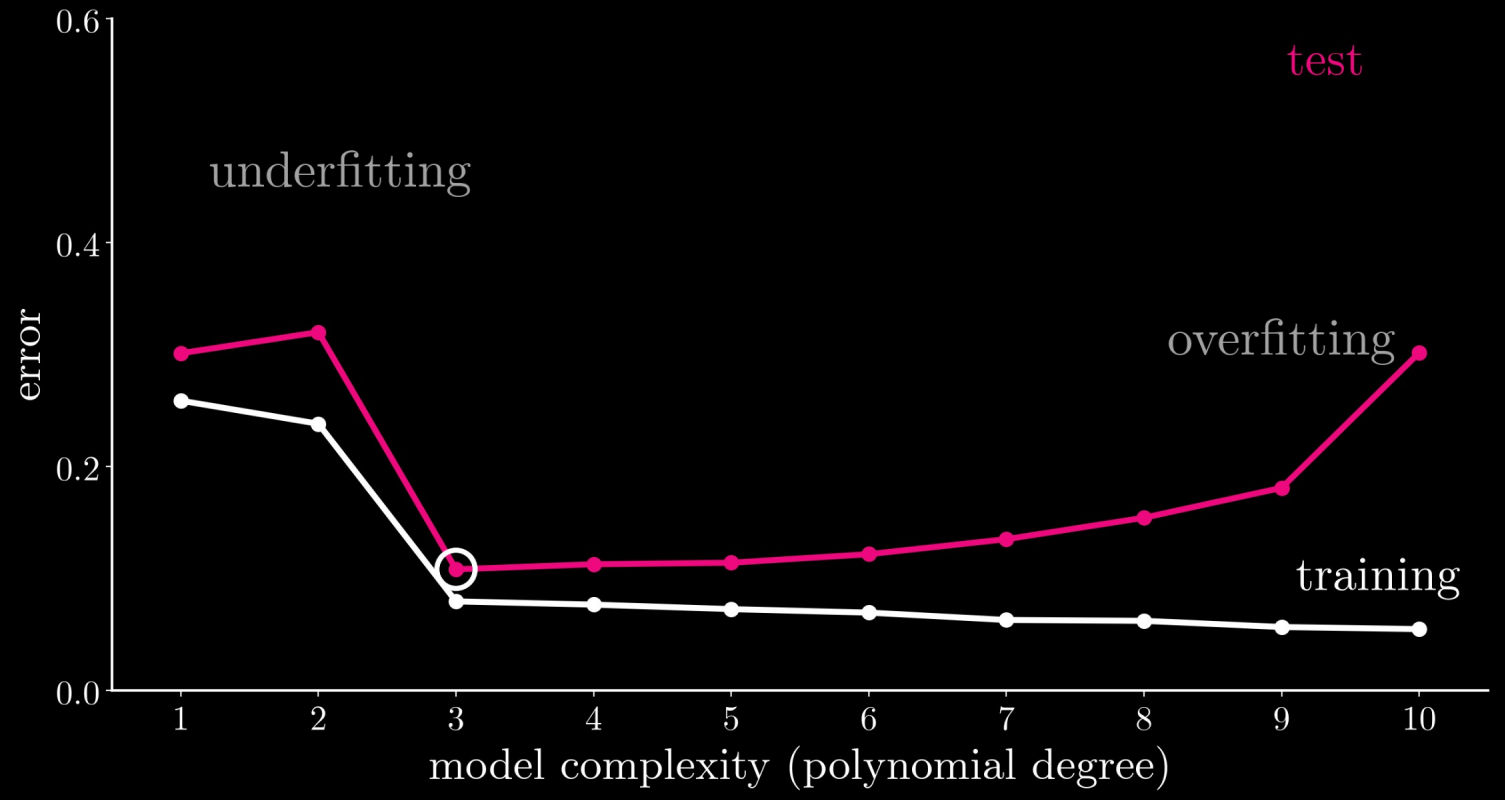
More flexibility always lowers the training error, but not the test error



Your turn: draw the test error in the air.

The U-Curve

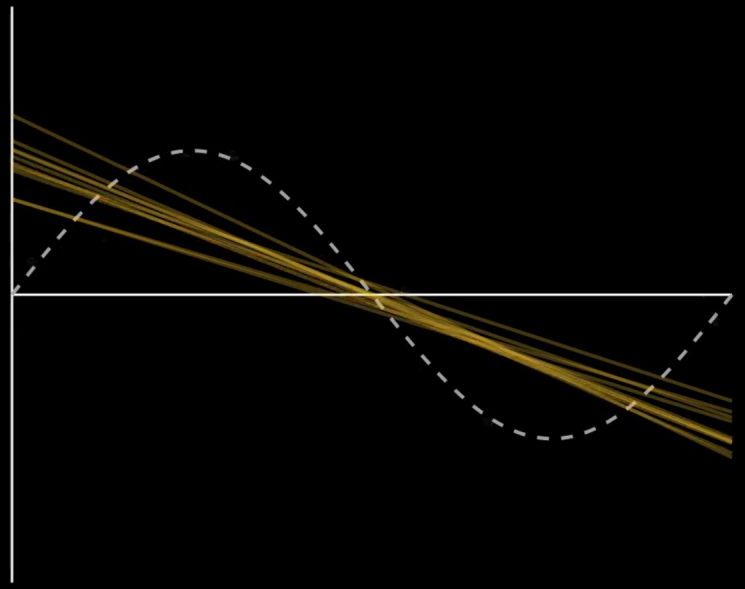
More flexibility always lowers the training error, but not the test error



Bias and Variance

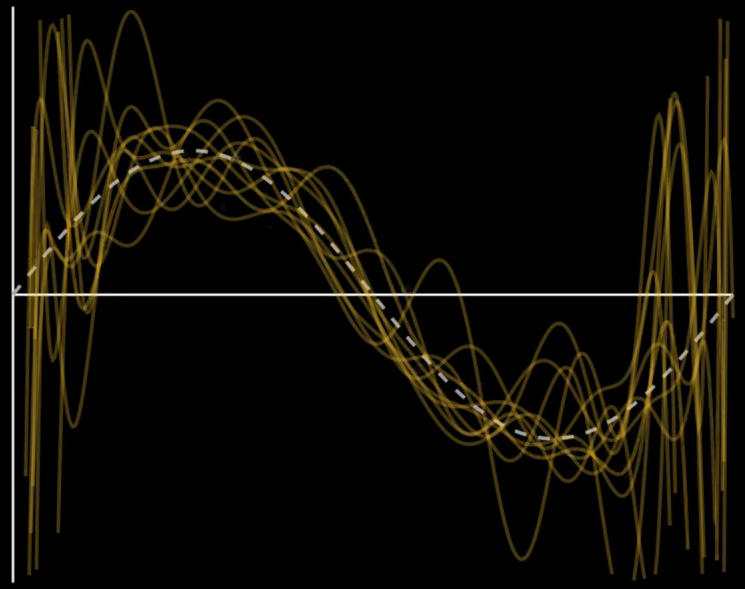
Why does the U rise? Because the training set itself is luck.

degree 1



bias: consistently wrong

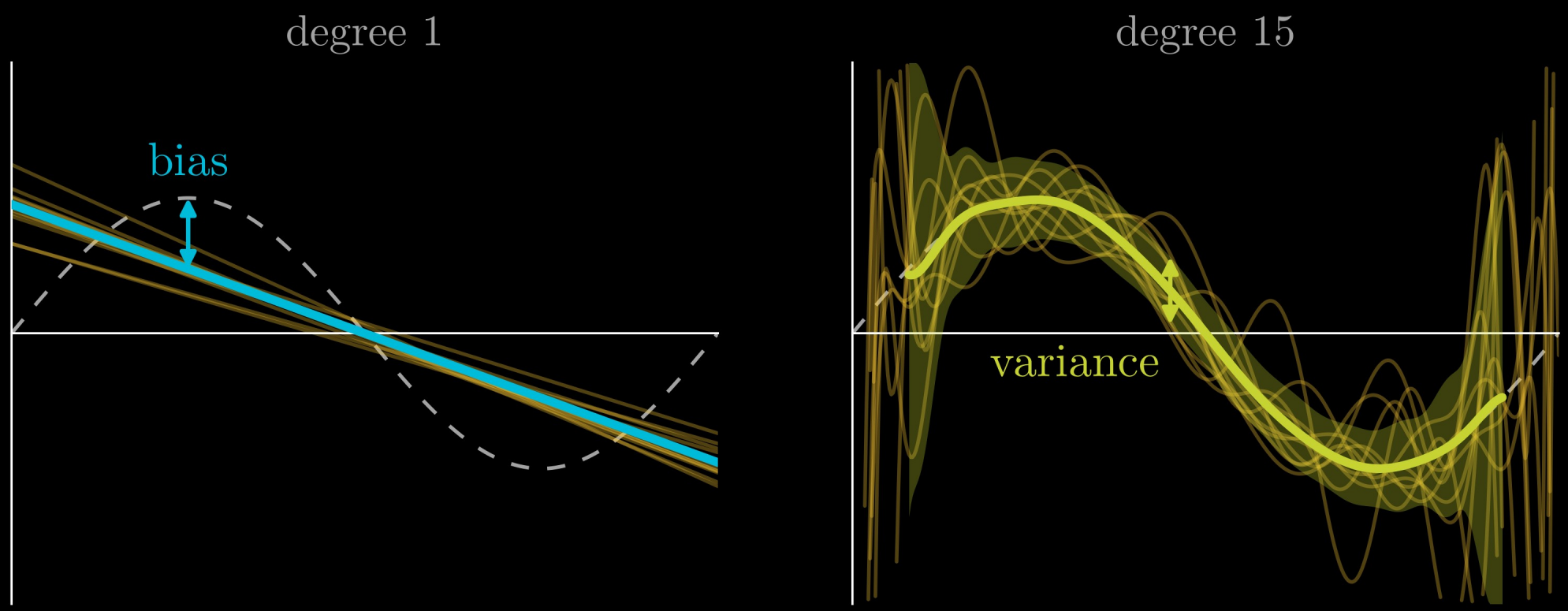
degree 15



variance: different every time

Average the Wobble

Your turn: average many wiggly fits. What do you get?



average many wobbly models and the wobble cancels: Thursday's random forests

The Trade-Off

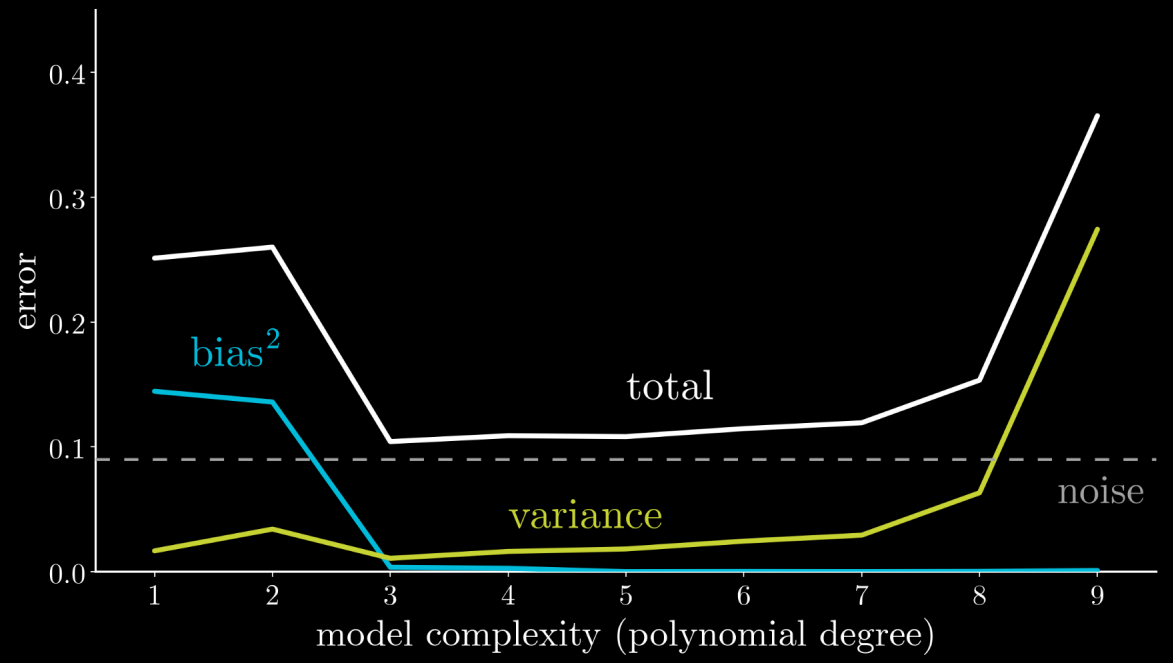
The expected test error splits into three parts

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \text{bias}^2 + \text{variance} + \text{noise}$$

bias²: average fit vs. truth

variance: scatter around the average

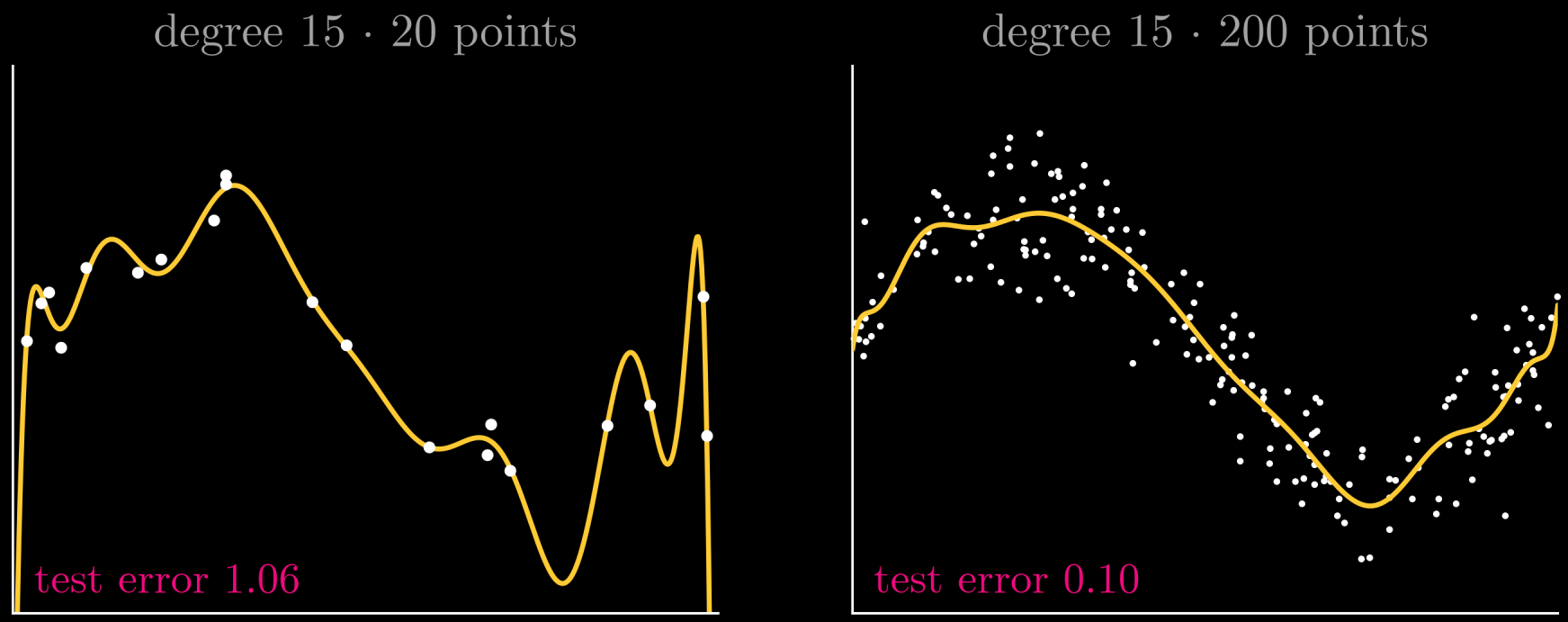
noise: scatter of y itself



curator's corner: wrong labels are noise too; they raise the floor no model can go below

More Data, Less Variance

Your turn: same degree-15 model, ten times more points. Does it still overfit?

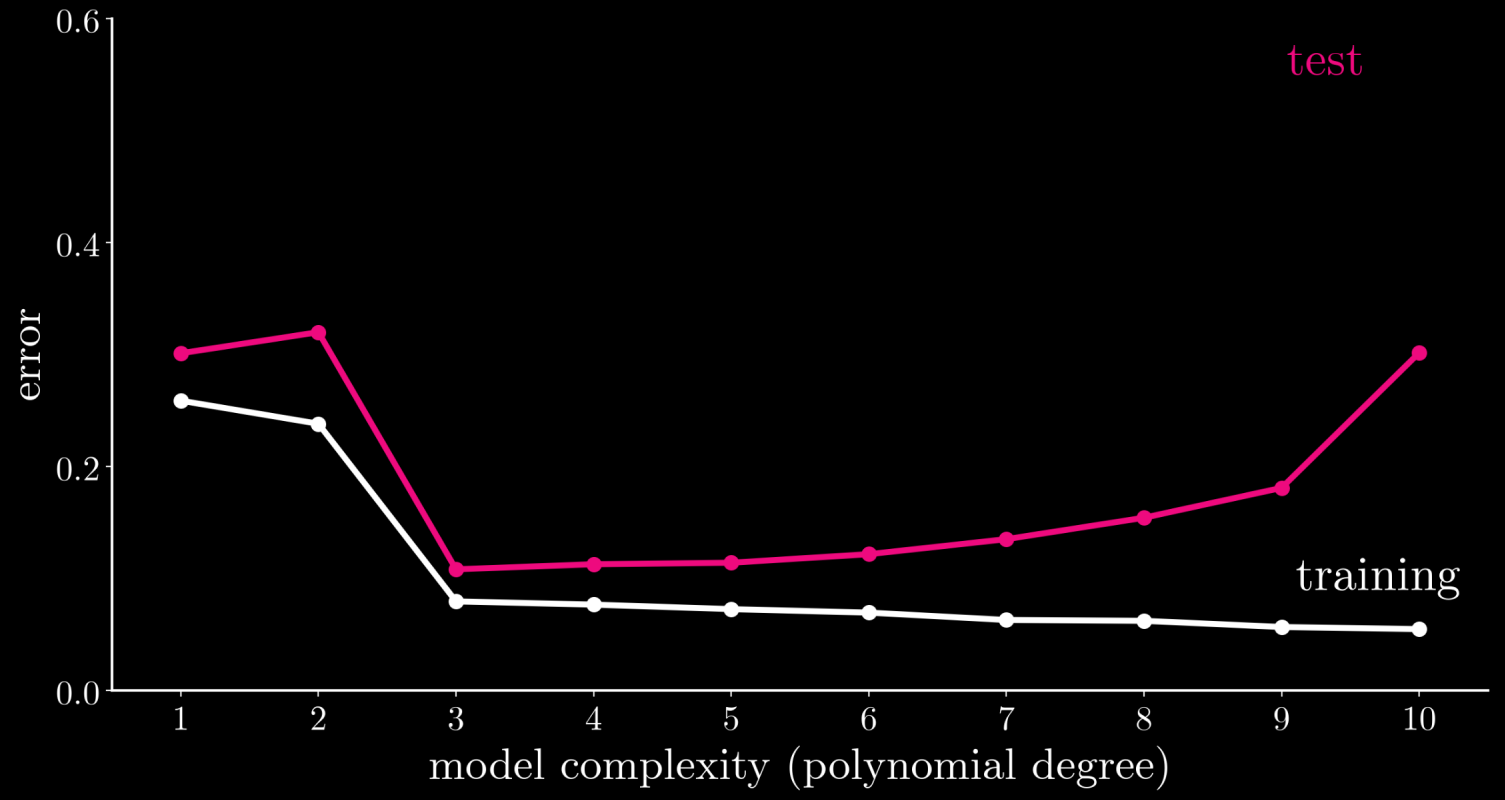


variance shrinks as the data grow: the bottom of the U moves right

curator's corner: the most reliable regulariser is more data, if it is good data

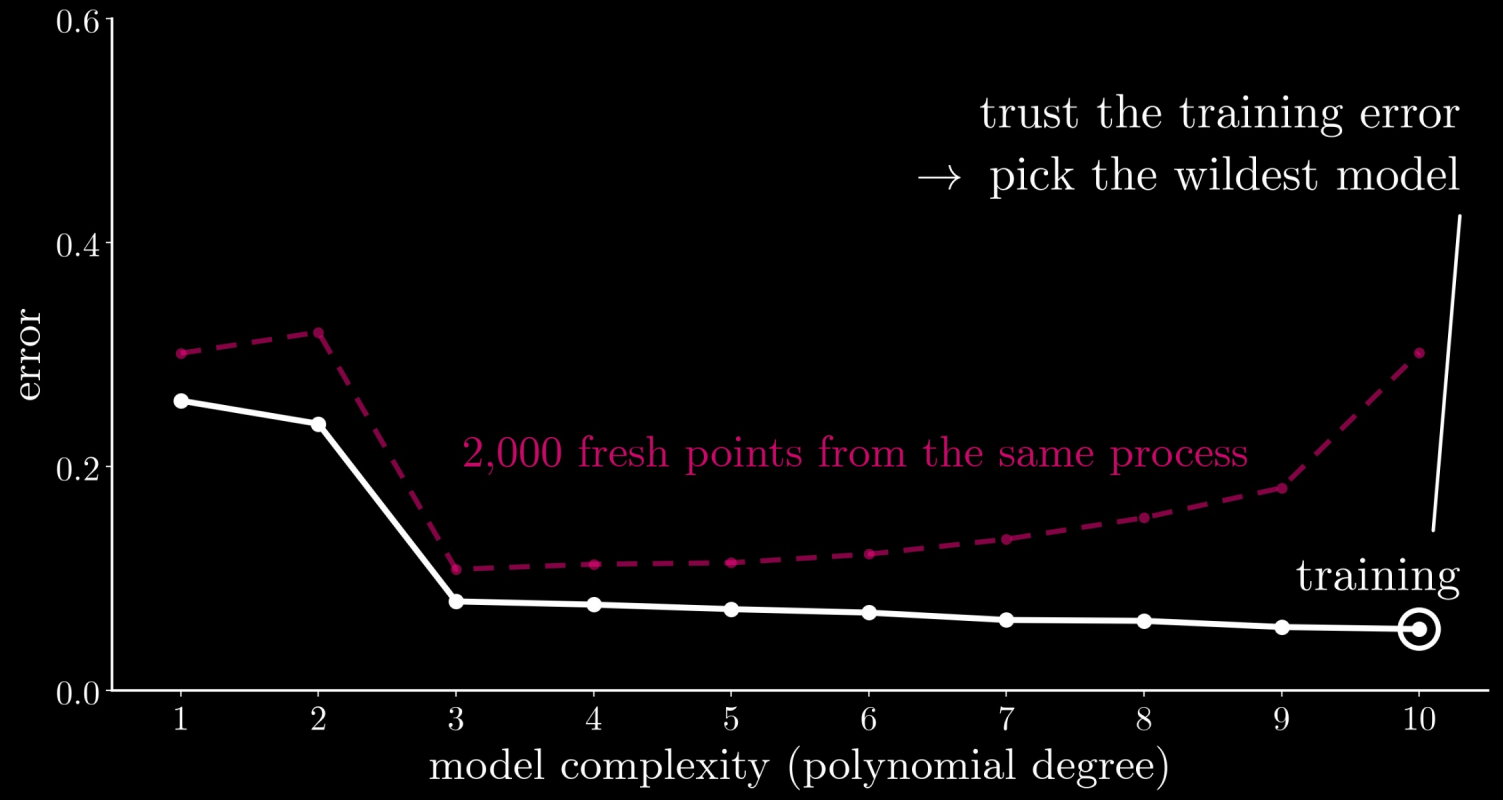
The Curve We Cannot Draw

The magenta curve needed 2,000 new points. In real life you only get the white one.



The Curve We Cannot Draw

The magenta curve needed 2,000 new points. In real life you only get the white one.



How can we see the magenta curve without new data?

Train, Validate, Test

How to measure the test error without cheating

Hold Some Data Back

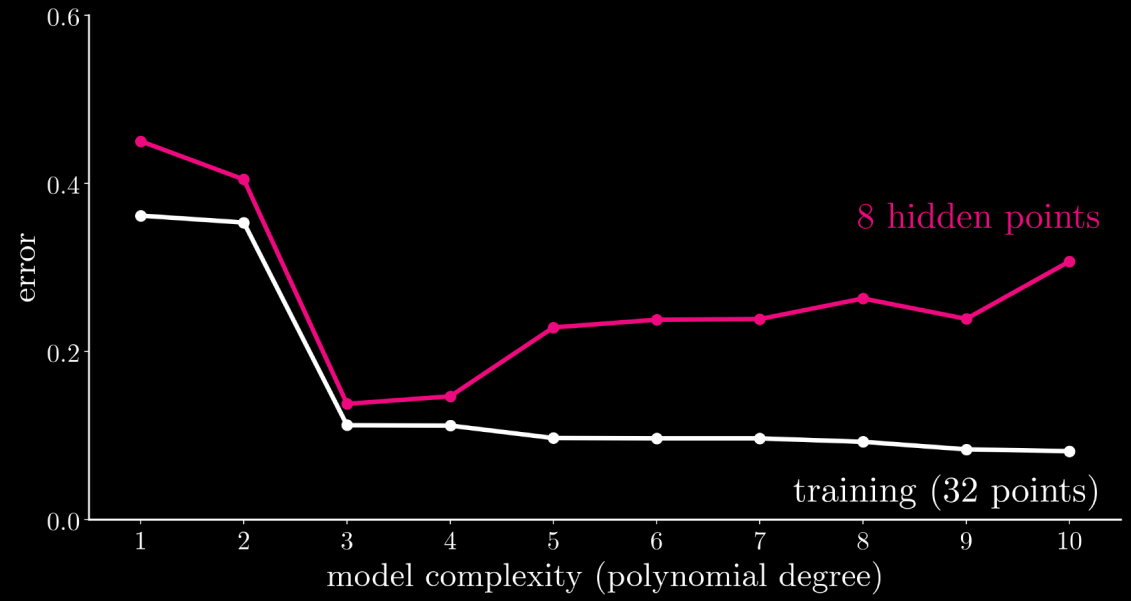
Hide part of the data from the model, and treat it as new

our 40 points



training: 32 points

test: 8 points, never shown to the model



curator's corner: split it off first, then forget it exists

Knobs and Settings

Parameters are learned; hyperparameters are chosen

parameters

learned by the optimiser,
on the training set

the weights w

the intercept b

hyperparameters

chosen by us,
not by the optimiser

the polynomial degree

the learning rate η

the penalty λ

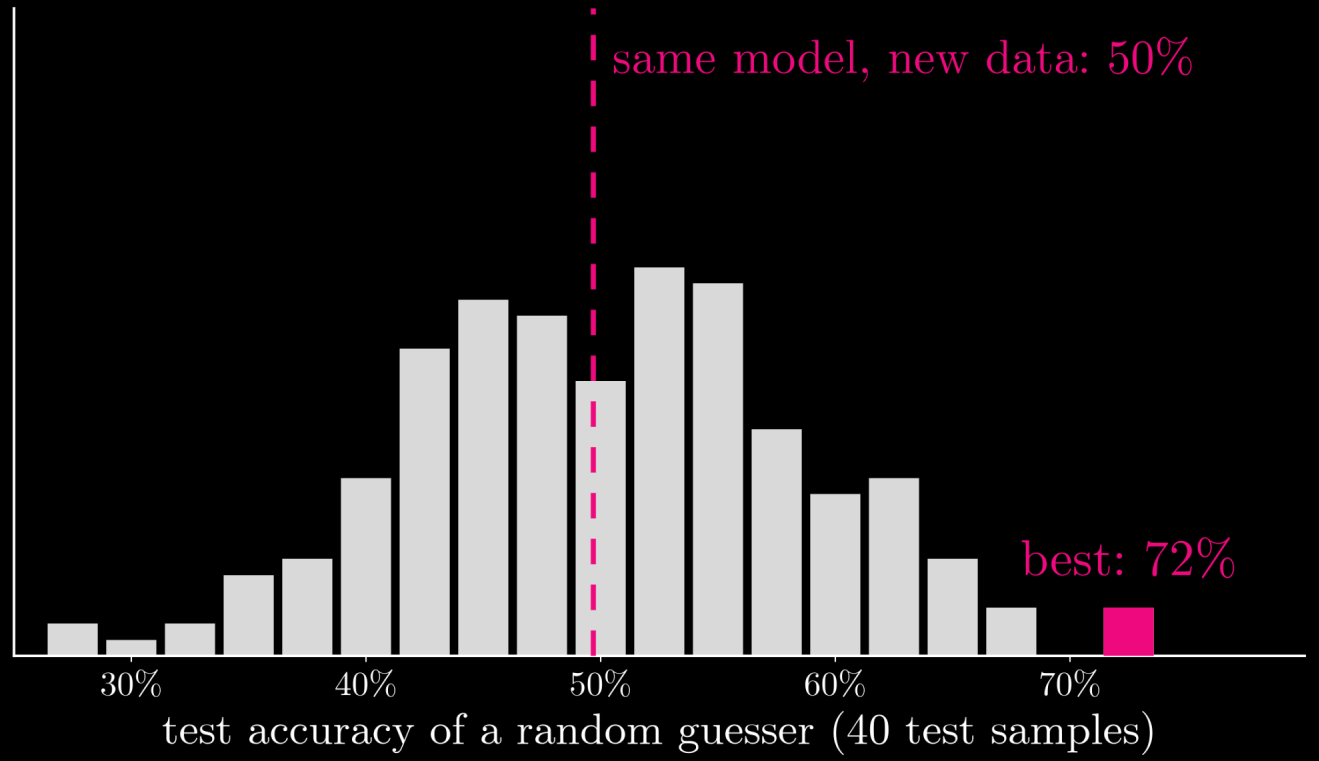
the k of k-nearest neighbours

Choose them on the training error, and you always pick the most flexible model.

On the test set, then?

Peeking at the Test Set

Your turn: 200 models that guess at random. How good does the best one look?



curator's corner: every look at the test set leaks a little of it into your choices

Three Sets, Three Jobs

A second held-out set, only for choosing



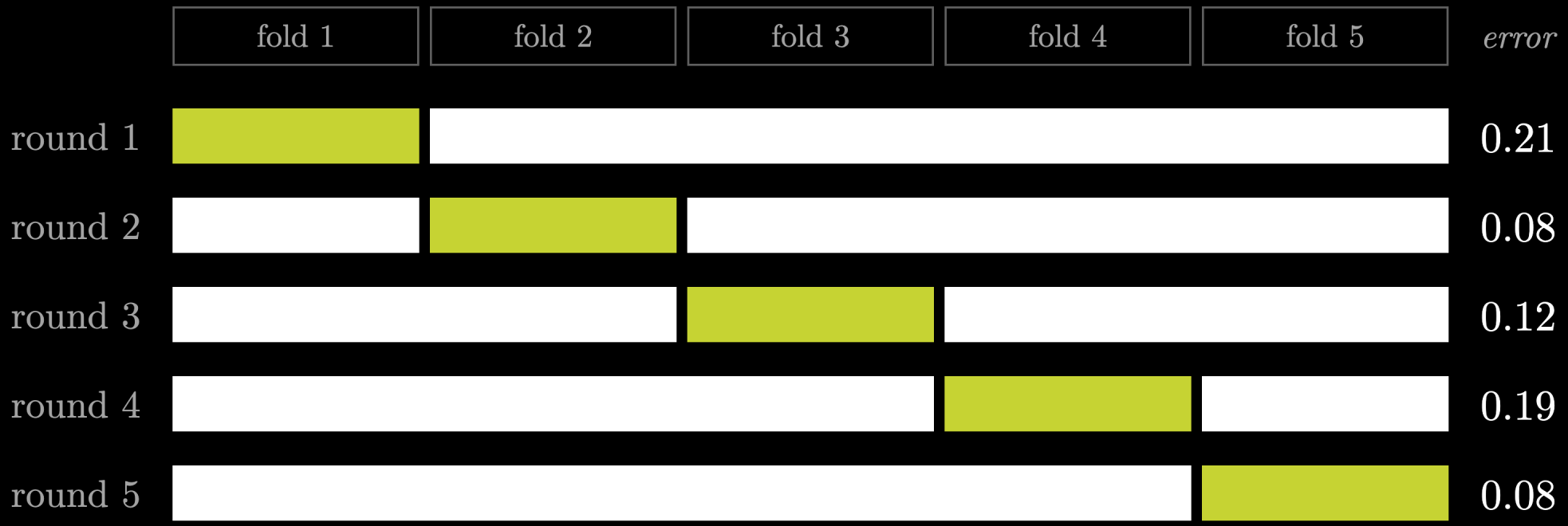
Three Sets, Three Jobs

A second held-out set, only for choosing



Cross-Validation

Do not trust one split: rotate the validation set

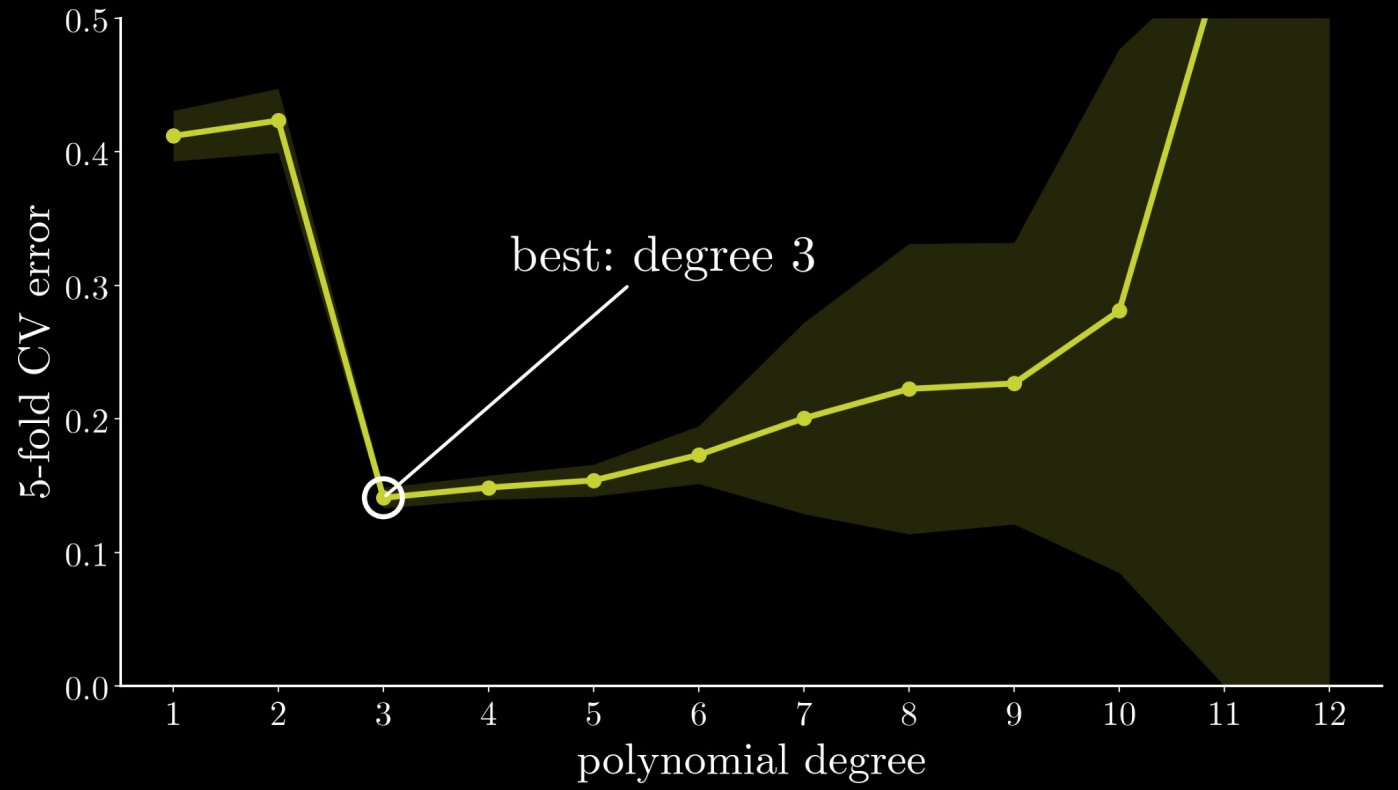


CV error = the average of the five = 0.14

training validation

Choosing the Degree

Pick the setting with the best cross-validated score



Then refit on all the training data, and test once.

Is 0.10 Good?

A score means nothing until you compare it with something

$$\text{test MSE} = 0.10$$

$\text{RMSE} = \sqrt{0.10} \approx 0.32$ in the units of y: off by about a third, on a wave from -1 to $+1$

the laziest model, always the mean of y: MSE $= 0.61$

$$R^2 = 1 - 0.10 \,/\, 0.61 = 0.84$$

the fraction of the variance explained: 0 = no better than the mean; it can go negative

the noise floor (0.09) caps it at 0.85

MAE, the robust cousin: Monday's outlier barely moves it

An Honest Workflow

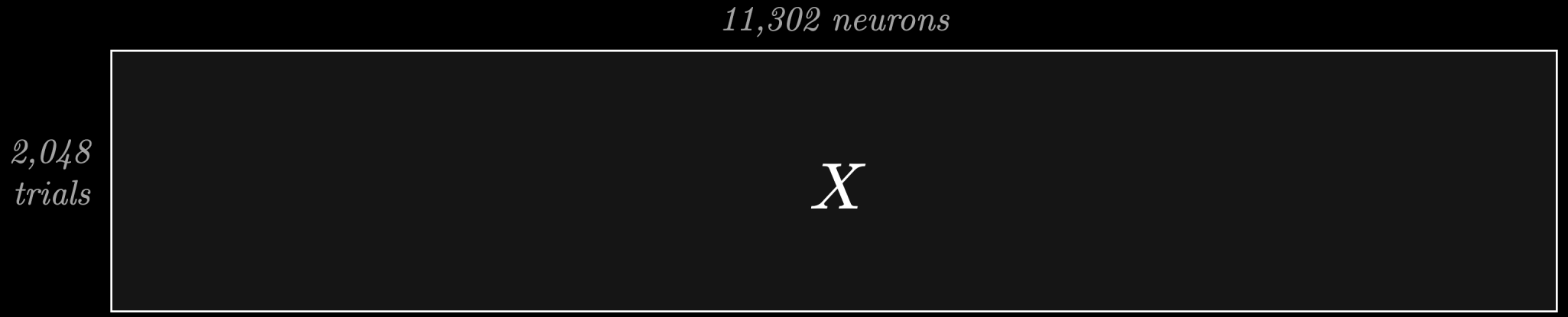
Five steps, and the test set is read exactly once



too few data for a test set? nested cross-validation runs this loop inside another one

More Knobs Than Data

On Thursday's neurons there is no degree to turn down



a linear decoder: one weight per neuron → 11,302 knobs for 2,048 trials

one knob per point was already enough to memorise anything

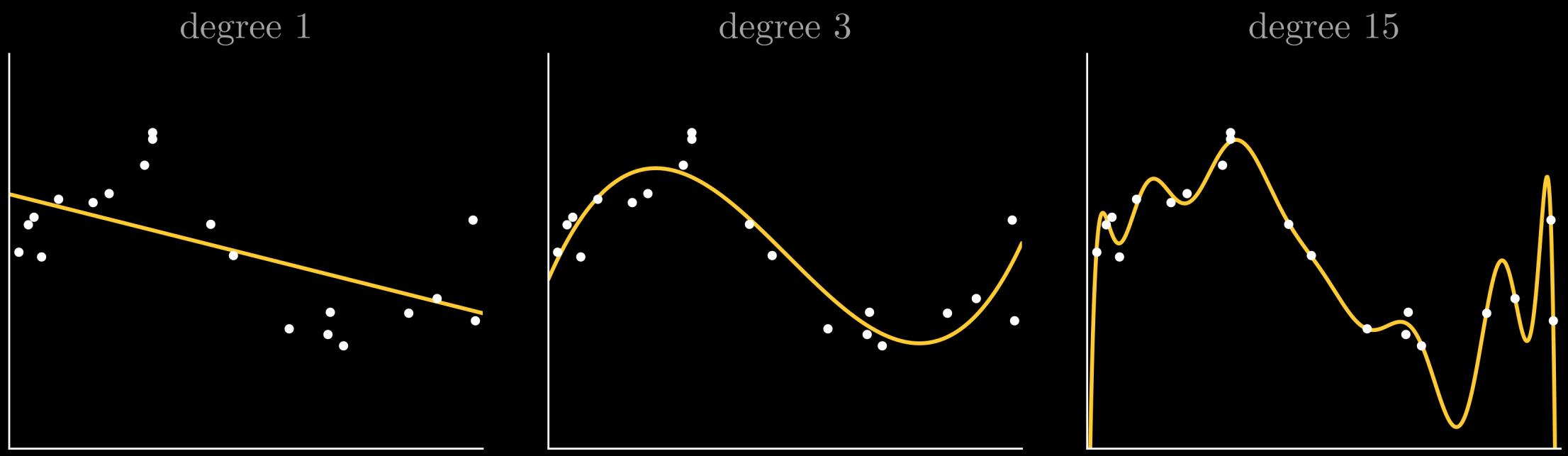
Drop neurons? Which ones? We need a knob that keeps every feature, but makes the model pay for using it.

Taming Flexibility

Keep every feature, but charge for using it

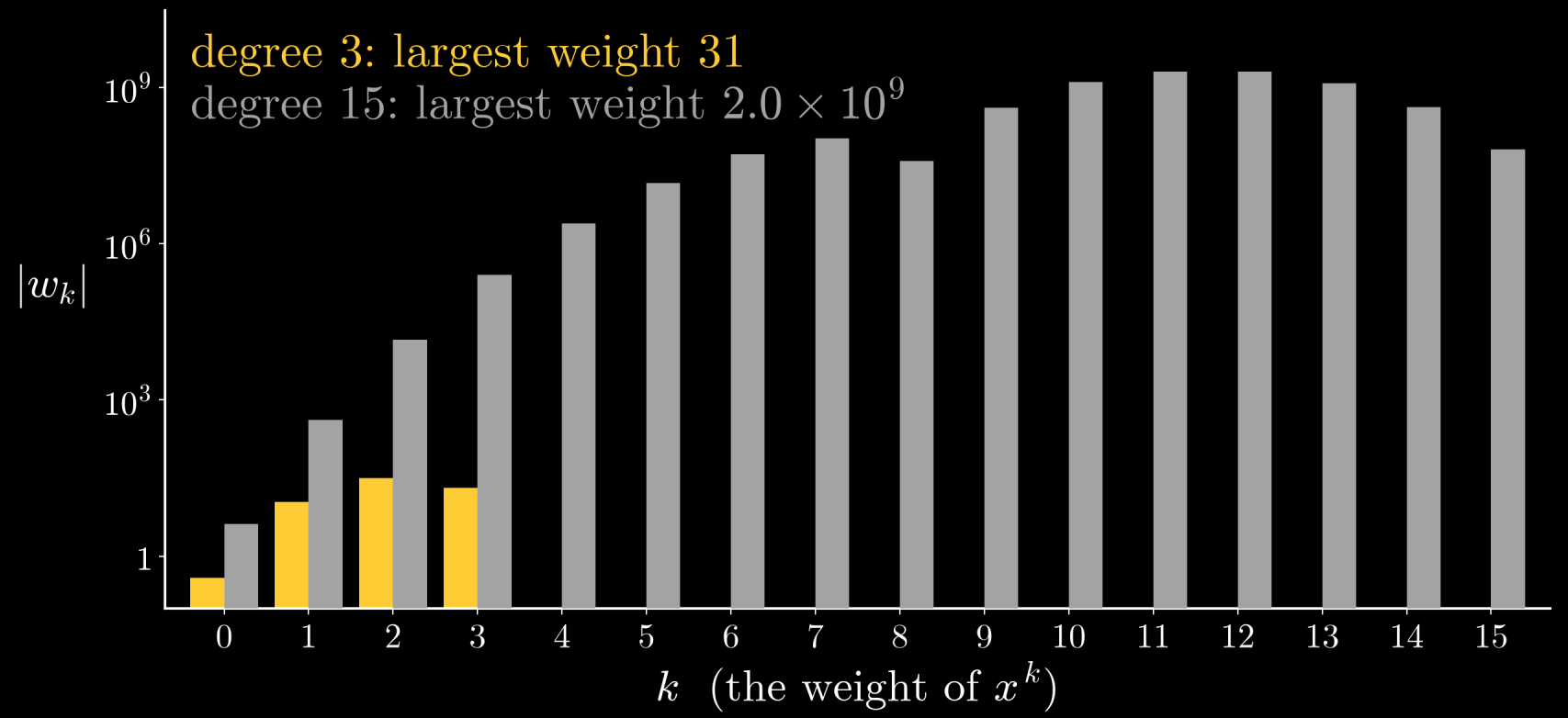
Look Inside the Wiggle

Your turn: both curves stay between -2 and 2. How big are their weights?



Look Inside the Wiggle

Your turn: both curves stay between -2 and 2. How big are their weights?



+2,000,000,000 here, −2,000,000,000 there: a wiggle is huge weights cancelling each other

so: charge for the size of the weights

Penalise Large Weights

Add a price for complexity to the loss

ridge

$$L(\mathbf{w}) = \frac{1}{N}\sum_i \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2 + \lambda \sum_j w_j^2$$

lasso

$$L(\mathbf{w}) = \frac{1}{N}\sum_i \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2 + \lambda \sum_j |w_j|$$

fit the data, but pay λ for every unit of weight

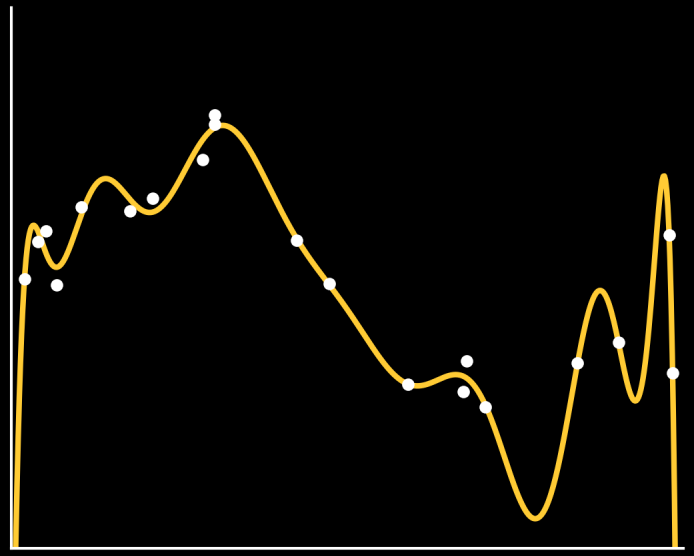
only the Loss row changes: the optimiser is still gradient descent

curator's corner: standardise first, or the units decide which weights get punished

Turning the Knob λ

Same degree-15 model, three values of λ

$\lambda \approx 0$

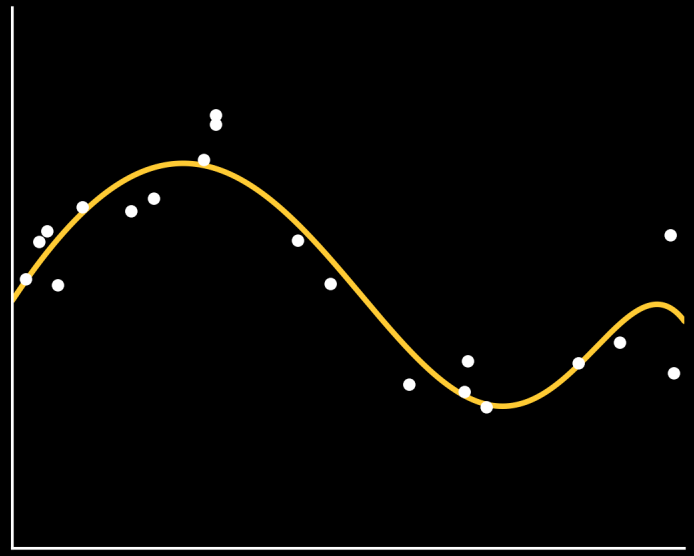


overfits, as before

test error 1.06

largest weight: 2,000,000,000

λ small

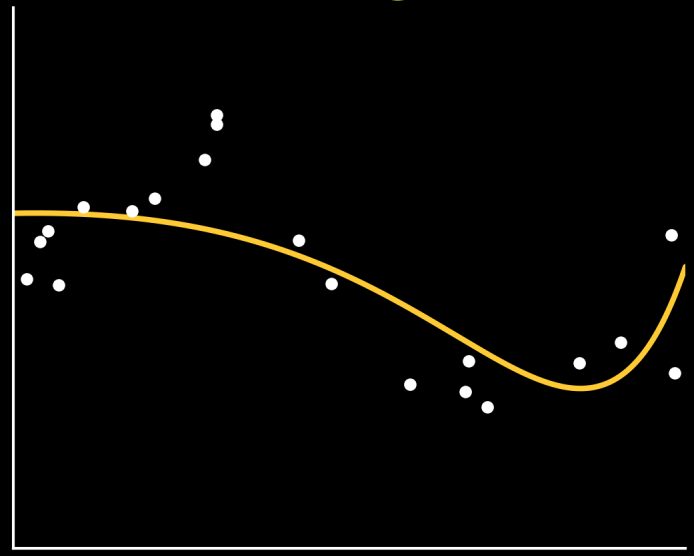


just right

test error 0.11

largest weight: 11

λ large



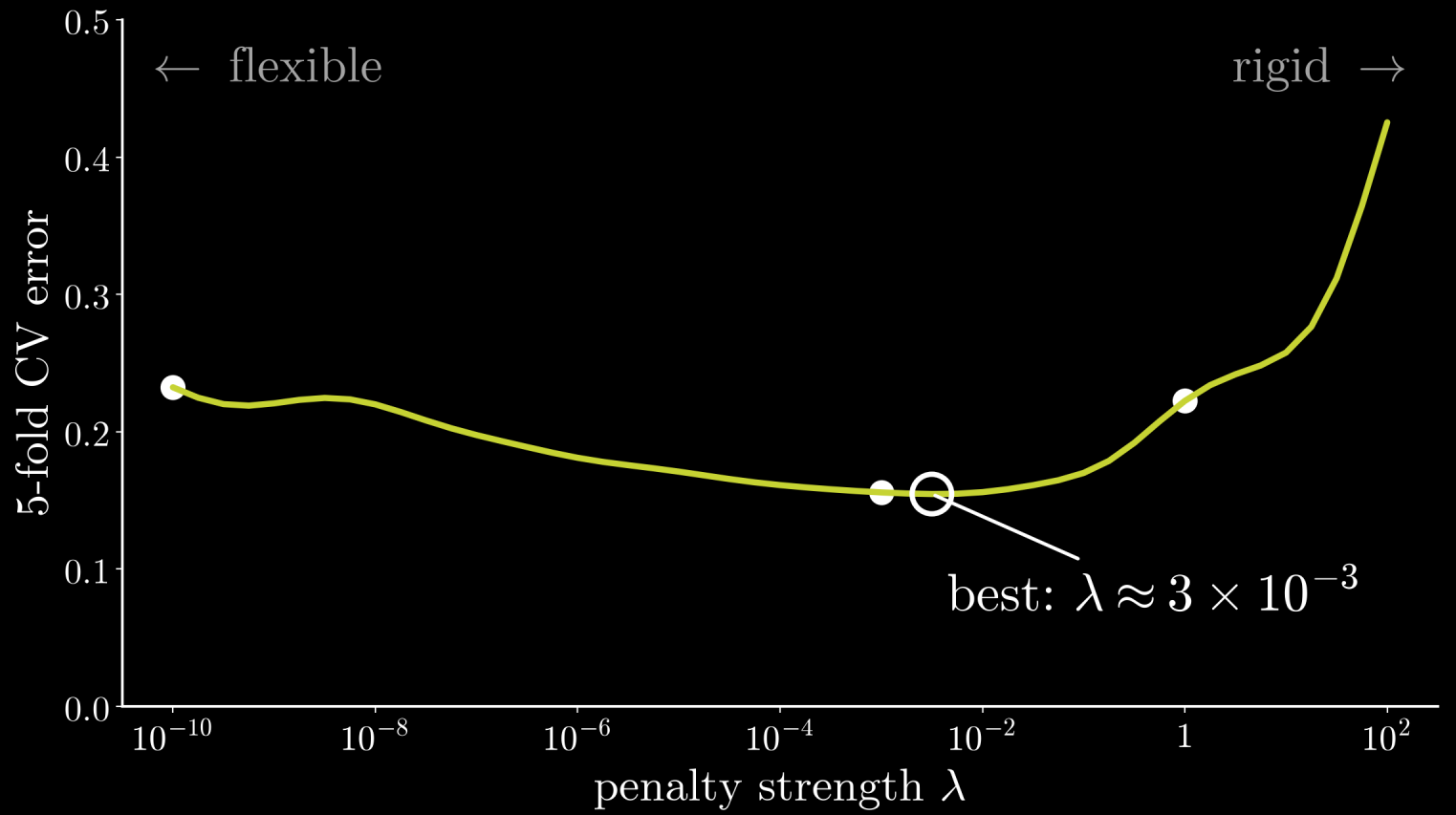
underfits

test error 0.21

largest weight: 1.4

Choosing λ

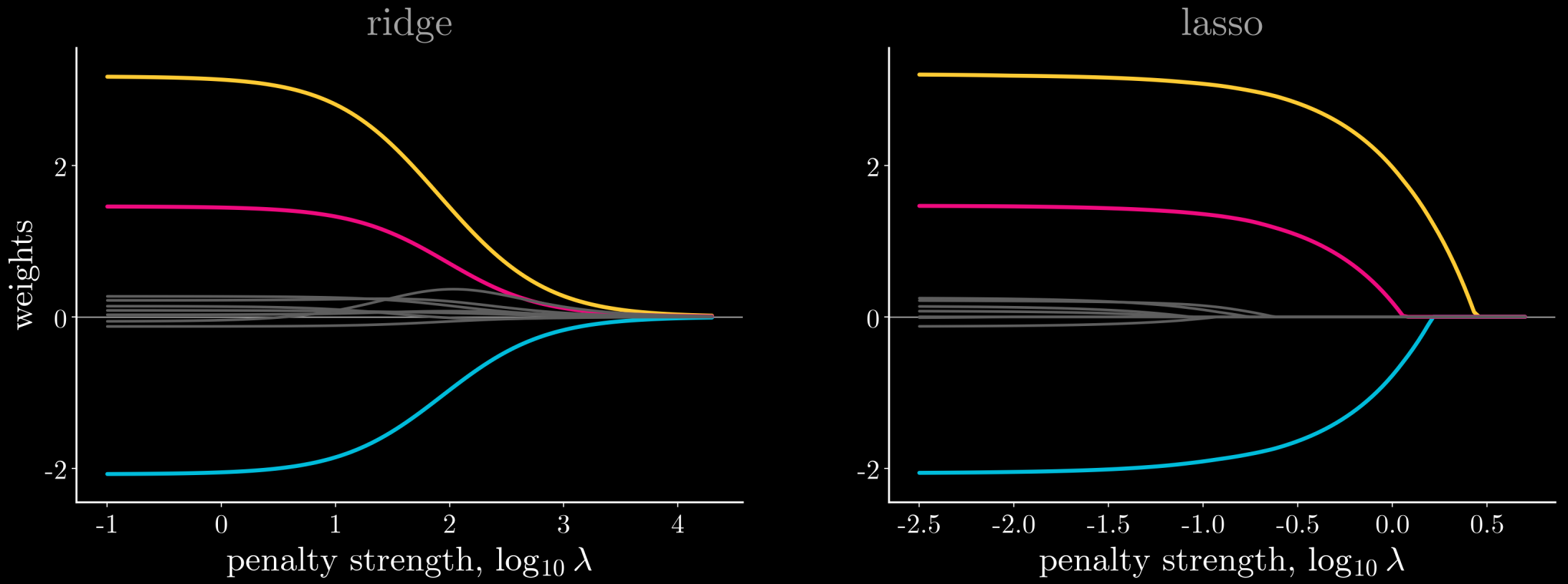
Your turn: on a λ axis, which side is overfitting?



scikit-learn's LogisticRegression is ridge-penalised by default, with $C = 1/\lambda$; Thursday's SVM has a C too

Lasso Picks Features

Your turn: ten features, three matter. As λ grows, what happens to the weights?

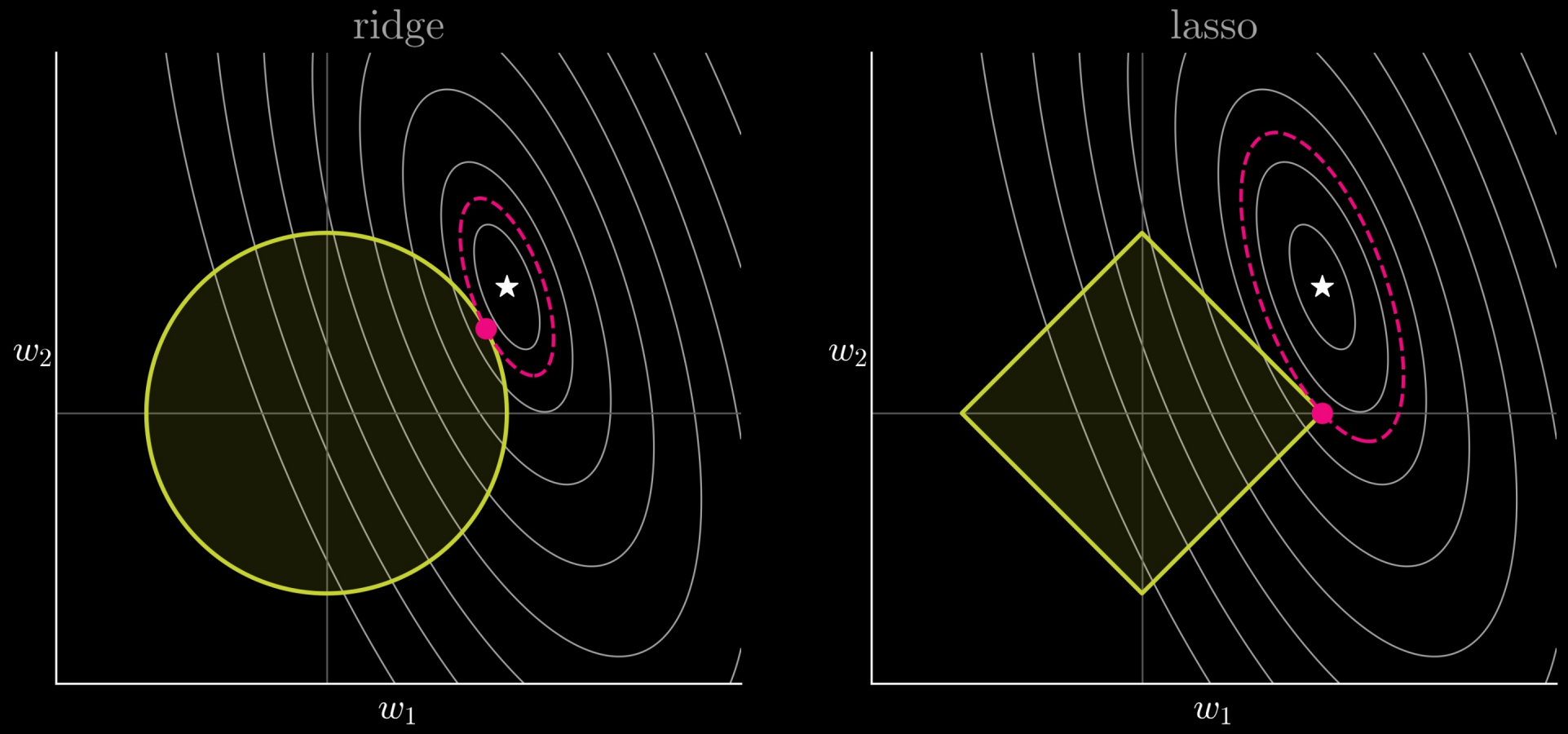


Will lasso's weights shrink together too?

a model that uses 3 features instead of 10 is also easier to explain

Ridge vs. Lasso

Why exactly zero? The shape of the penalty decides.



corners sit on the axes: lasso sets some weights exactly to zero

Where We Stand

An honest procedure. Two questions left.

which number?

data → split → choose → refit → test once → report



degree, λ

$R^2 = 0.84$

can we trust the data?

Break

Over coffee: a test for a rare disease is 99% accurate. Would you take it?

Keeping Score

Which number to report, and what to compare it with

A 99% Accurate Test

Your turn: a test for a rare disease is 99% accurate. Is it good?

1 person in 100 has the disease.

A test that always says "healthy" is 99% accurate.

On imbalanced data, accuracy tells you almost nothing.

curator's corner: the 1% is a fact about how the sample was collected; record it

Always Compare to a Baseline

Before celebrating, ask what a trivial model would score

task	*trivial model*	*its score*
classification	always the majority class	accuracy 99% (our test)
regression	always the mean of y	$R^2 = 0$ (this morning)

If your model cannot beat these, it has learned nothing.

in scikit-learn: DummyClassifier, DummyRegressor

on Thursday: chance is 1 in 32; DeepLabCut's baseline is a second human

The Confusion Matrix

Count the four ways a yes/no prediction can go

	predicted sick	predicted healthy
actually sick (100)	80 *true positives*	20 *false negatives*
actually healthy (9,900)	300 *false positives*	9,600 *true negatives*

Your turn: is its accuracy higher or lower than the lazy test's 99%?

accuracy 96.8%: worse than always saying "healthy", yet it finds 80 of the 100 sick

with 32 directions it becomes a 32 × 32 table: Thursday

Precision and Recall

Two questions, two numbers

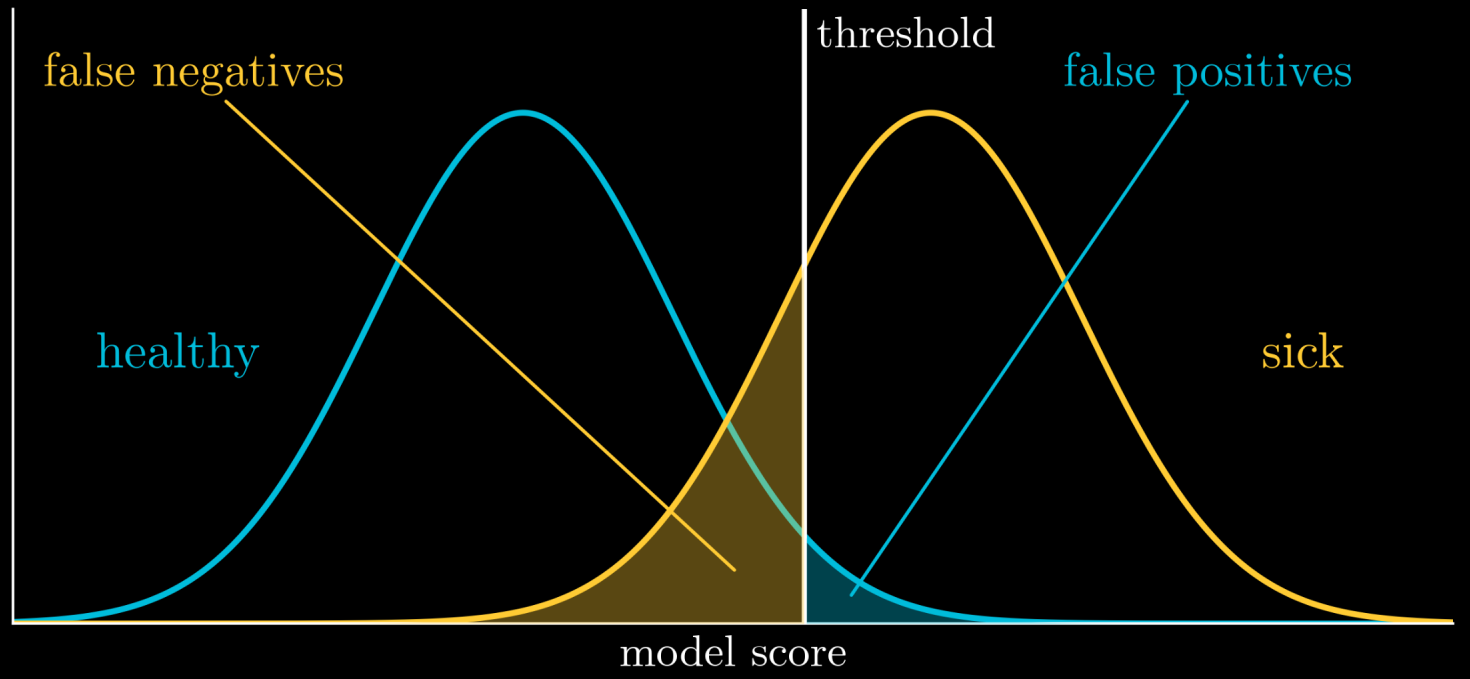
of the alarms, how many were real? $$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{80}{380} \approx 21\%$$

of the real cases, how many did we catch? $$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{80}{100} = 80\%$$

high only if both are $$F_1 = 2\,\frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \approx 0.33$$

Moving the Threshold

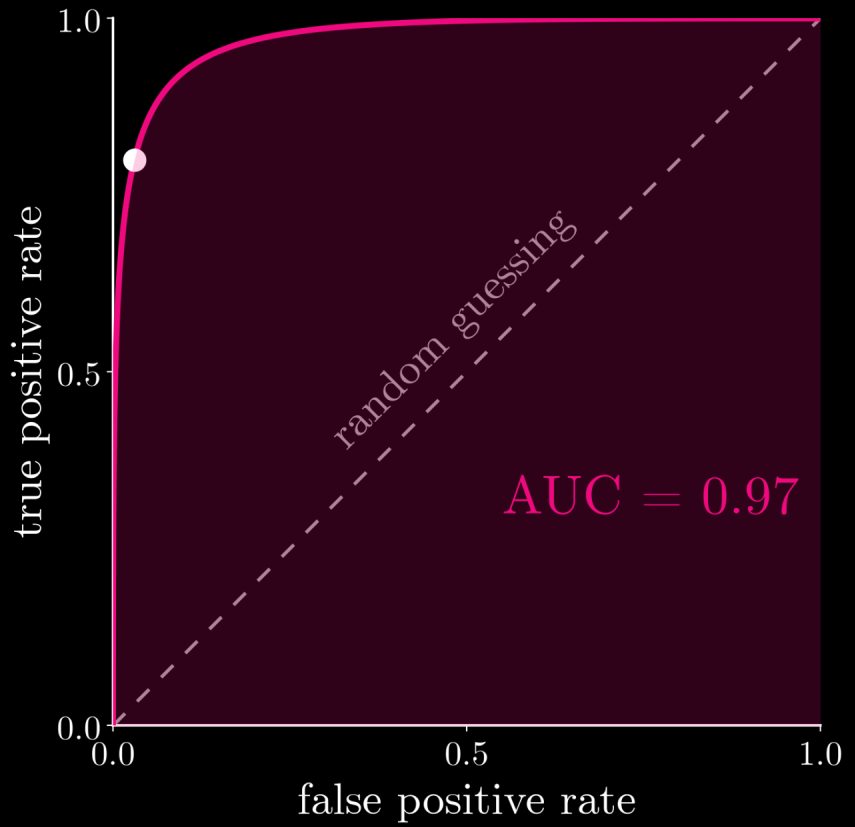
A classifier outputs a score; we choose where to cut



Move the line right: fewer false alarms, more missed cases.

ROC and AUC

Sweep the threshold; plot what you catch against what you wrongly flag



the diagonal: random guessing

Every threshold is one point on the curve.

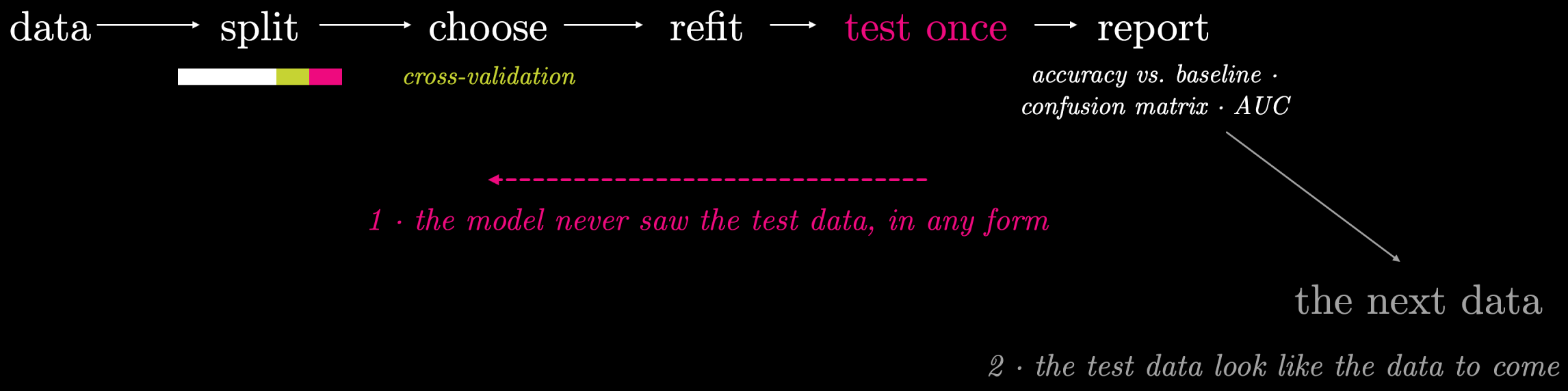
the white dot: the matrix we just read, catching 80% of the sick and flagging 3% of the healthy

AUC = 0.97: the chance that a random sick person scores higher than a random healthy one

0.5 = guessing, 1 = perfect

What Every Score Assumes

Every number so far trusted the test set twice

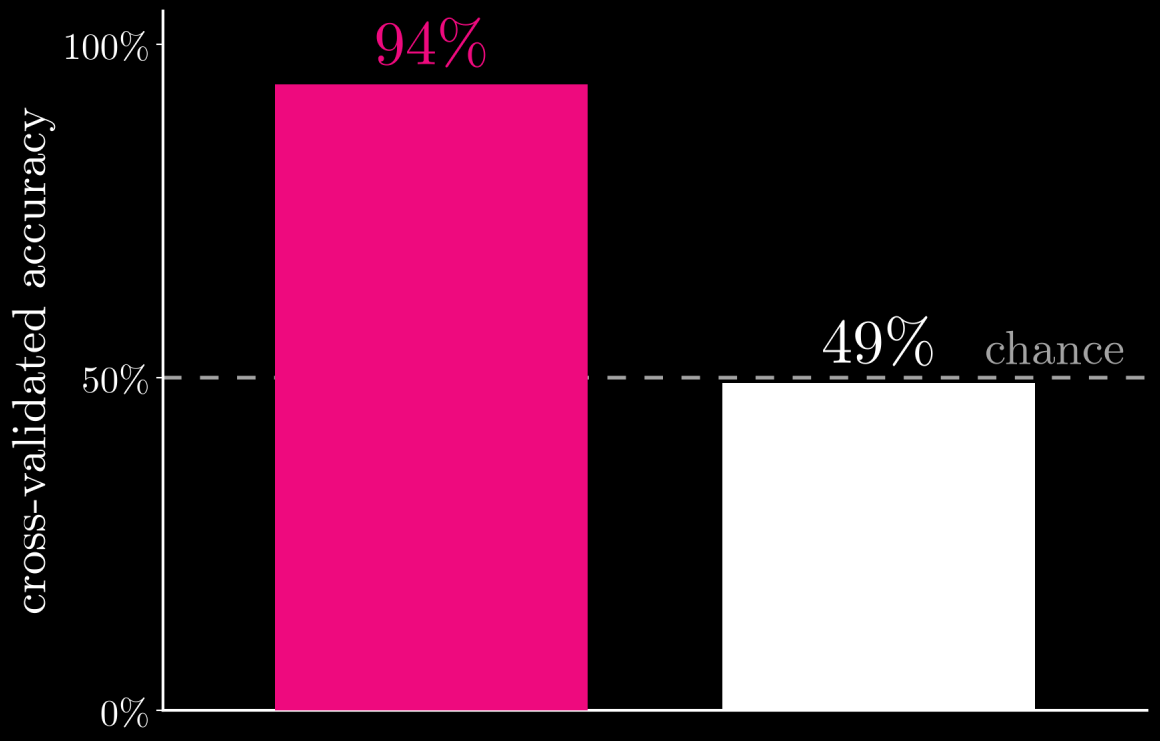


When the Data Lies

When the test set is not new, or not like the future

A Leak in Action

Your turn: pure noise, 50 samples, 5,000 features. What accuracy do we get?



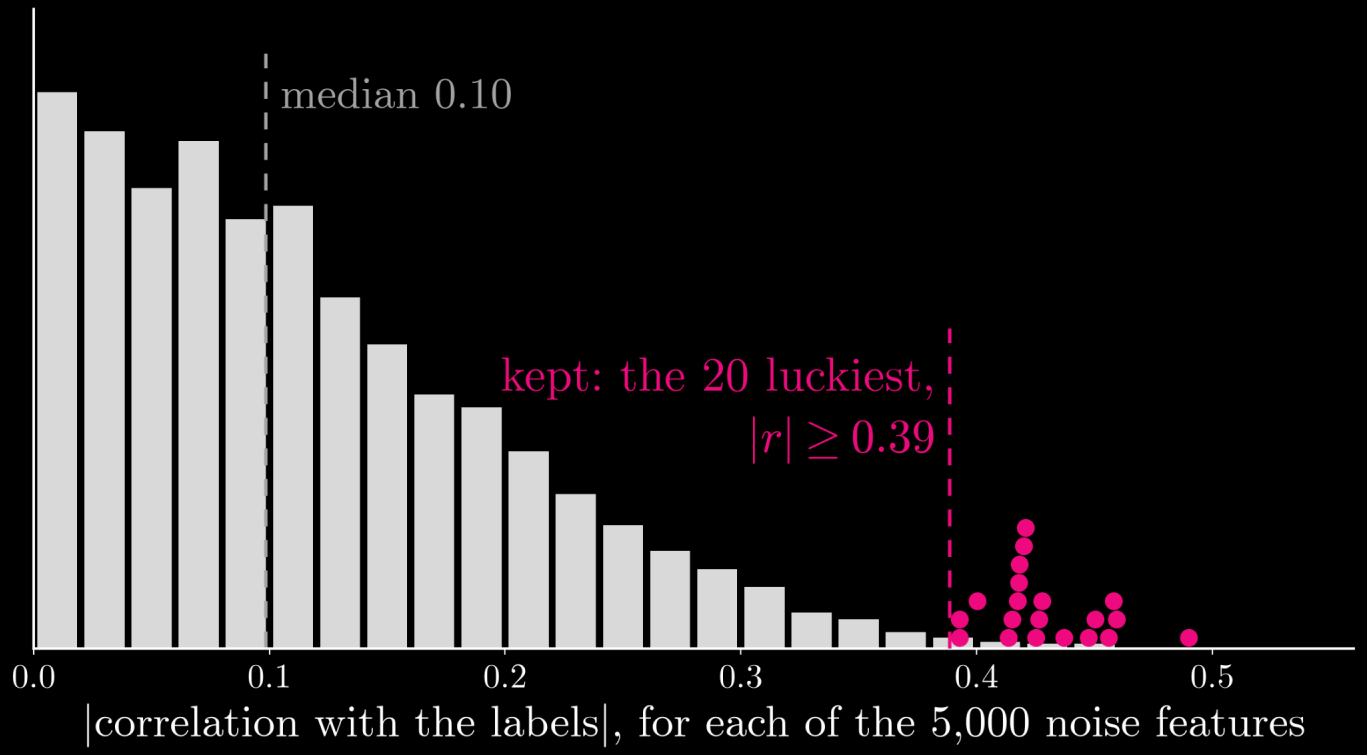
select the 20 best features on all the data, then cross-validate

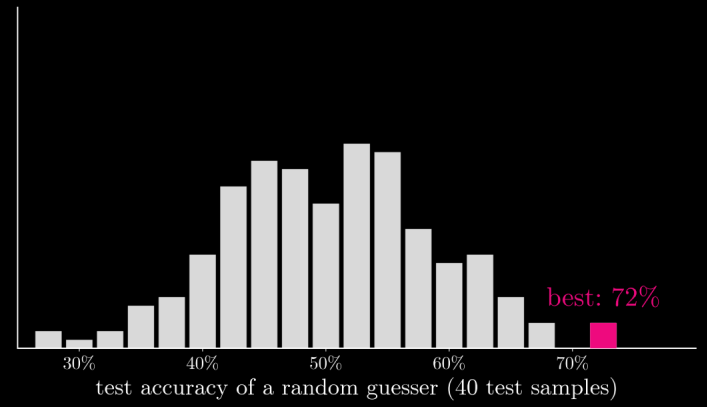
select the features inside each fold

curator's corner: every step that looks at the labels belongs inside the cross-validation loop

Peeking in Disguise

5,000 noise features are 5,000 coin flips; the selector kept the luckiest





this morning: the 72% coin

Leakage is peeking you did not notice.

The Fix: Pipelines

Bundle preprocessing and model, so cross-validation refits everything on each fold

```
pipe = make_pipeline(StandardScaler(),
                     SelectKBest(k=20),
                     LogisticRegression())

scores = cross_val_score(pipe, X, y, cv=5)
```

On each fold: fit the scaler, the selector and the model on the training part only; then score the held-out part.



Leaks No Pipeline Can Catch

Some leaks live in the code; most live in the data

preprocessing fitted before the split → *a pipeline*

duplicates across the split → *deduplicate, then split*

the same record from two sources; consecutive frames of one video

information from the future → *split by time*

in a time series, a random split lets the model peek ahead

the same animal, session or patient on both sides → *split by group*

trials from one session share the drift, the mood, the lighting

Ask what will be new when the model is used, and make that the unit of the split.

Clever Hans

A model can be right for the wrong reasons

A horse that could "count"...

...by reading tiny, involuntary cues from the person asking.

An image classifier that could recognise horses...

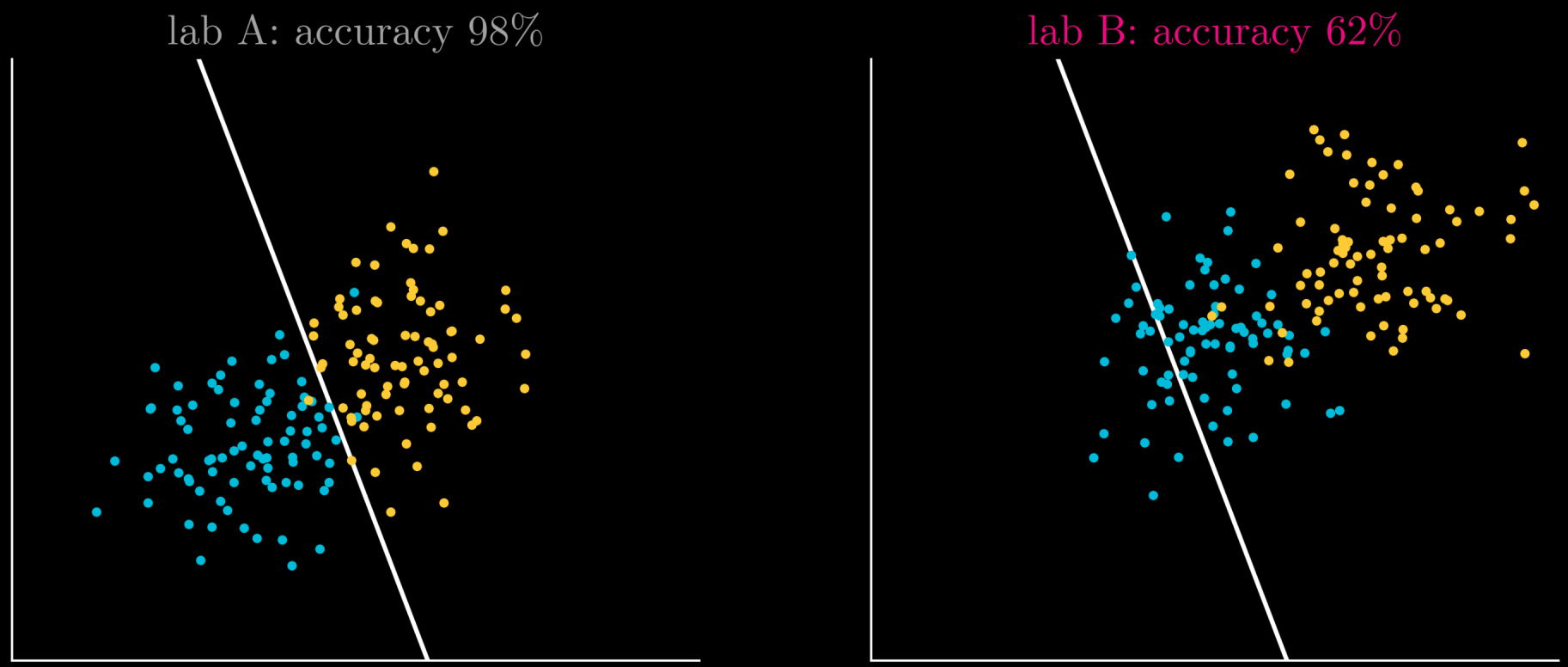
...by a copyright tag in the corner of the horse photos.

Take the questioner out of sight, and the counting stops.

Pfungst 1911; Lapuschkin *et al.*, *Nat. Commun.* 2019

Distribution Shift

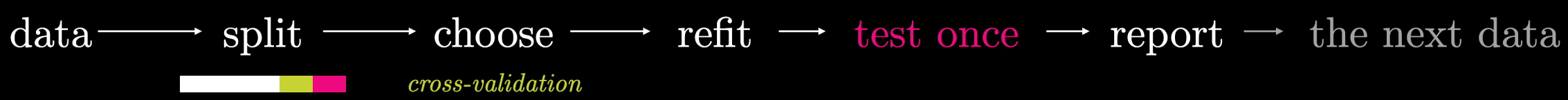
The world the model meets is not the world it learned from



a test set from the same lab will not warn you

A Curator's Checklist

Seven checks before you believe a number



1 split first; test once

2 split by animal, session, patient

3 look for duplicates

4 fit preprocessing inside the folds

5 compare to a baseline

6 read the confusion matrix, not just the accuracy

7 ask where it will come from

curator's corner: data quality decides whether you can believe the number

Hands-On 2

Overfit on purpose, then learn not to fool yourself

Core	Exercises 1–5: two sets, two errors, the U-curve, one split's opinion
Stretch	Exercises 6–9: cross-validation with a pipeline; choose the degree and λ; test once
Challenge	Exercises 10–11: find the leak, then the honest number (take the checklist with you)

`github.com/myscience/intro-ml-mdmc-2026`

notebooks › N2_not_fooling_yourself.ipynb › Open in Colab

The Recipe, Revised

Same four ingredients, one new rule

Data $(x_i,\ y_i)$ inputs and their answers

split first, by the unit that will be new; beware leaks and shift

Model $f(x;\ \theta)$ a function with knobs

flexible enough, and no more: bias against variance

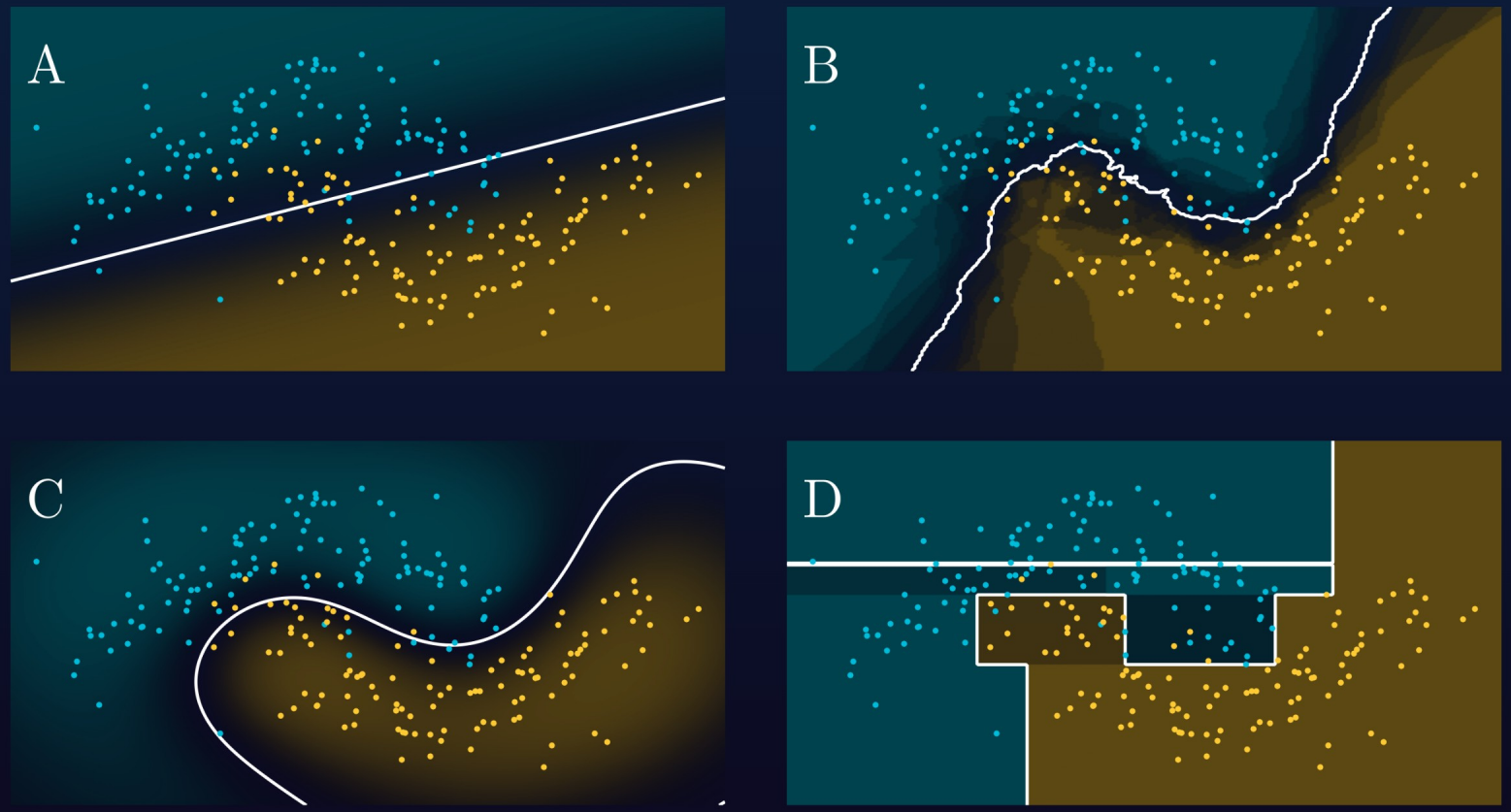
Loss squared error, cross-entropy $+\ \lambda \cdot$ penalty

Optimiser gradient descent

Judge every model on data it has never seen.

On Thursday

Same data, four models, four different boundaries



Which one is the most flexible? Which would you trust? You already know how to decide.

which model drew which? On Thursday: beyond the straight line

Introduction to Machine Learning · Day 2

Thank You

Thursday: Beyond the Straight Line